Qiskit Hack Day | Tokyo · dual-track year trial | seed 42 · fully reproducible

Q Grid

A hybrid quantum–classical dispatch layer that tells every EV in the city **when and where to charge** — at the exact optimum, verified.

github.com/Aghosh-mv/Q-Grid | JustCode × Qub3RAM.team | Apache-2.0 + addendum

30/30 QAOA EXACT-OPTIMUM CELLS

65× QAE QUERY REDUCTION

6 q 48-SLOT PROFILE ENCODING

0 CAPACITY VIOLATIONS · YEAR

02 · PROBLEM STATEMENT

Urban EV charging is a congestion problem with a price on every minute.

Tokyo's network is a patchwork of small stations. At midday, **solar glut makes power nearly free**; evenings price it at a premium. Yet drivers arrive when *they* are free, cluster at the nearest hub, and queue — cheap renewables curtail unused while peakers fire at dusk.

In our simulated Tokyo year the fleet grows **48 → 180 EVs on the same 11 stations**, and naive first-come-first-served dispatch leaves money on the table in every contested window — worst case **38.23% above the exact optimum**.

We optimize **assignment + scheduling**: which station, which slot, per car — minimizing grid cost + driver travel + waiting under strict port capacity. A city-scale **QUBO**: the class where exact search explodes 2^n and quantum earns its seat.

WHO PAYS TODAY

🚗 **Drivers** — waiting instead of driving

🔌 **Operators** — congested hubs, peak-priced energy

🌏 **The grid** — noon solar curtailed, dusk peakers fired

THE COMBINATORIAL WALL

$2^{20} = 1{,}048{,}576$

schedules on one 20-qubit crunch cell — **doubles with every extra car**.

03 · PROPOSED SOLUTION

One software agent per car. One center for the city.
Four solvers, one referee.

INPUT →

Telemetry

Position, SoC, arrival window per EV — consent-gated, ward-coarsened, DP-noised.

PROCESSING

AI triage → QUBO

SoC<25% → NOW · <40% → SOON · else CAN-WAIT (₹260/₹150/₹80 per-slot wait weight). Classical pre-filter keeps 2 nearest stations; capacity is **slack-encoded as an exact equality** — no penalty on empty ports, overfill impossible.

→ OUTPUT

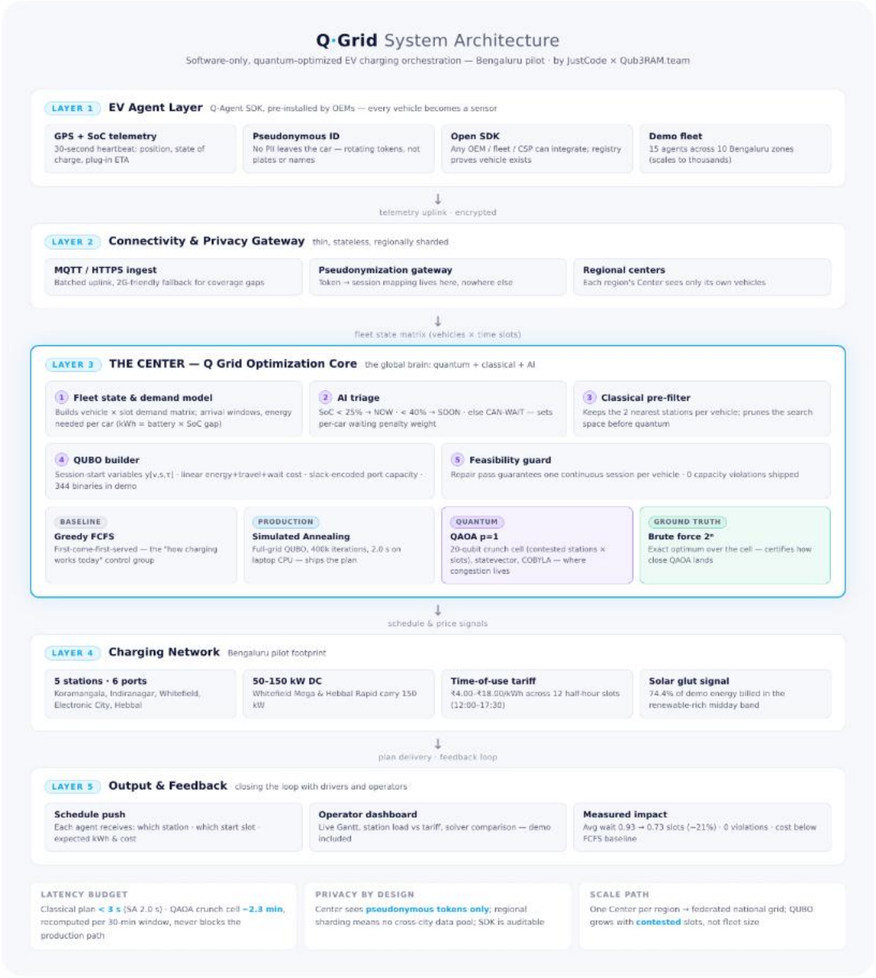
Final schedule

One continuous session per car, capacity-safe by construction. Deferred-load queue absorbs overflow.

SOLVER	ROLE	SCOPE
Greedy FCFS	baseline — what operators run today	full grid
Simulated Annealing	production path · 400k iters	full grid (344 QUBO binaries)
QAOA p=1 (COBYLA, 8k shots)	**quantum attack on the crunch cell** — most contested stations × slots	20-qubit subproblem
Exact brute force	referee — ground truth	crunch cell (2^{20})

04 · ARCHITECTURE

Classical pipes, quantum punches.



OEM software agent — no hardware retrofit · **ward-level DP telemetry** · **hierarchical crunch-cell detection** · **deferred-load queue**

05 · QUANTUM COMPONENT (MANDATORY)

QAOA on the crunch cell + amplitude encoding + QAE

1 · QAOA P=1

Mapping: session-start binaries y[v,s,τ] → Ising Hamiltonian; cost layer = QUBO, mixer = RX ring.
Why: combinatorial constrained dispatch is QAOA's home turf; p=1 + COBYLA certifies against brute force.

Achieves: exact optimum on **30/30** audit cells.

2 · AMPLITUDE ENCODING

A 48-slot Tokyo tariff/solar profile = 64 amplitudes = **6 qubits** (naive: 48 wires). Exact prep at the Shende–Bullock–Markov bound: **47 Ry + 57 CNOTs, fidelity 1.000000**. Zero-padding 48→64 costs nothing.

3 · QUANTUM AMPLITUDE ESTIMATION

QPE-based QAE (m=6 control) estimates renewable coverage at MC accuracy with **63 oracle calls vs 4,096 MC samples** — **65× query reduction**, measured at three anchors.

⚖️ **The honest ledger:** amplitude encoding compresses *data*, not decision variables — the QUBO stays plan-compressed at 11–20 qubits. Three levers, each labeled: **504 naive → 11–20 plan-compressed → 6 data qubits**. Not "the whole problem fits in 6."

06 · QISKIT USAGE

Native Qiskit 2.6, cross-validated.

SRC/QISKIT_BRIDGE.PY — RUNS IN 2 S

```python
# the same 6-qubit Tokyo profile, rebuilt in Qiskit
from qiskit.circuit.library import StatePreparation
from qiskit.quantum_info import Statevector
from qiskit.primitives import StatevectorSampler

qc = QuantumCircuit(6)
qc.append(StatePreparation(amps), range(6))
fid = abs(np.vdot(amps, Statevector(qc).data))**2
shots = StatevectorSampler(seed=42).run([qc]).result()
```

fidelity 1.000000 | **4,096-shot TVD** 0.033 | **qiskit** 2.6.1

LIBRARIES & ALGORITHMS

QuantumCircuit · **StatePreparation** · **Statevector** · **StatevectorSampler**

QAOA (p=1, COBYLA, 8k shots) · QPE-based QAE (m=6) · exact state prep (SBM bound)

HYBRID LOOP

Classical pipeline builds the QUBO → ships the **crunch cell** to QAOA → sampled bitstrings decoded to schedules → **30-cell audit** scores all four solvers on identical instances.

BACKEND — STATED PLAINLY

Statevector simulation (NumPy + Qiskit primitives). No physical QPU run — documented as limitation #19, not hidden.

Slack-encoded capacity: the detail that makes annealing actually work.

```
min  Σ_v,s,τ  y[v,s,τ]·( E(v,s)·tariff(τ)
              + 9·dist(v,s) + w(v)·(τ − arr_v) )
s.t.  Σ_s,τ  y[v,s,τ] = 1            # each car charges once
      Σ_v x[v,s,τ] + u[s,τ] = ports(s)  # EXACT equality
```

The slack binaries **u** (one per free port) turn the capacity inequality into an **exact equality** — so no penalty is ever applied to *empty* ports, and the solver can never overfill a station. This is why the entire simulated year closed with **0 capacity violations**.

WHY THIS ENCODING

Penalty-based soft constraints trade violation risk for hyperparameter tuning. Slack encoding removes the tuning and makes feasibility structural.

CRUNCH-CELL DETECTION

Contention-scoring auto-picks the most contested stations × slots (e.g. Koramangala Hub + Indiranagar Point, 12:00–12:30, 4 lowest-SoC cars) — quantum effort goes where it pays.

THREE QUANTUM BUGS WE FIXED

① bitmask leak → full higher-bit context ② S_0 in wrong basis → Q=−I, all estimates 1.0 ③ L1 vs L2 — fidelity 0.045 = $\|p\|^2$ was the clue. Full log →

One Tokyo year, run twice. 16 minutes.

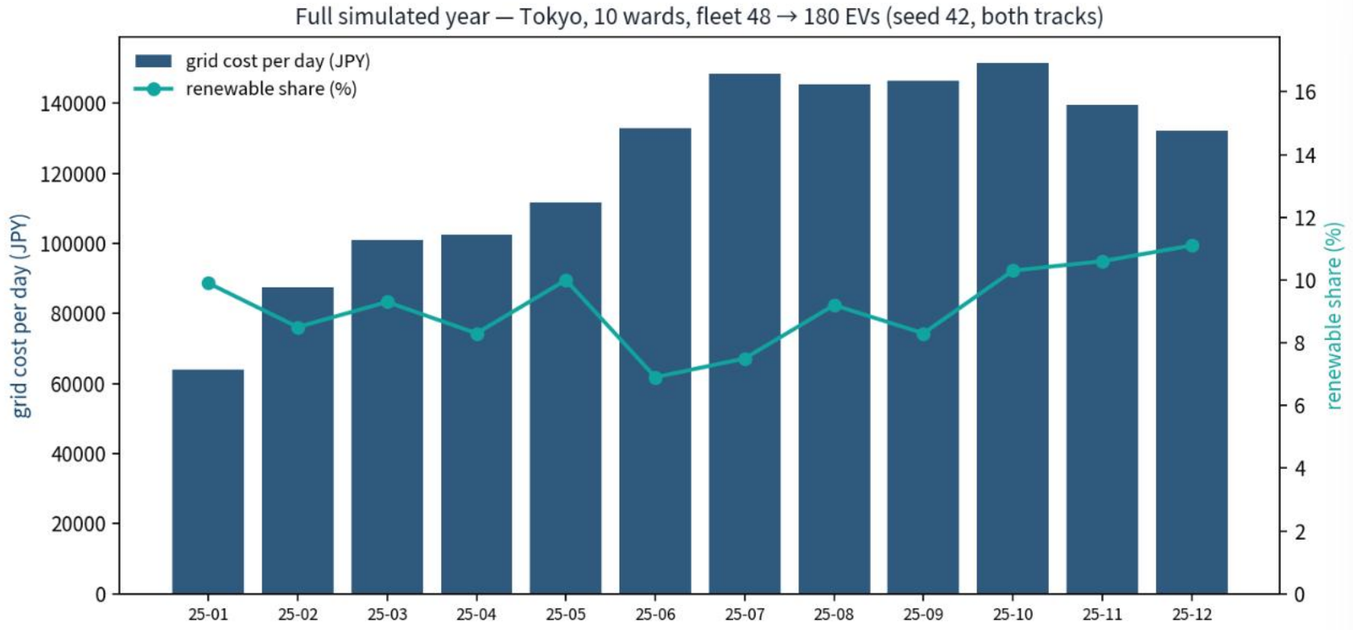
991 s
365 DAYS · 31,826× COMPRESSION

45,766
CHARGING SESSIONS

1.0 GWh
1,000,759 KWH DELIVERED

¥44.5M
GRID COST · JPY TOU

0
CAPACITY VIOLATIONS

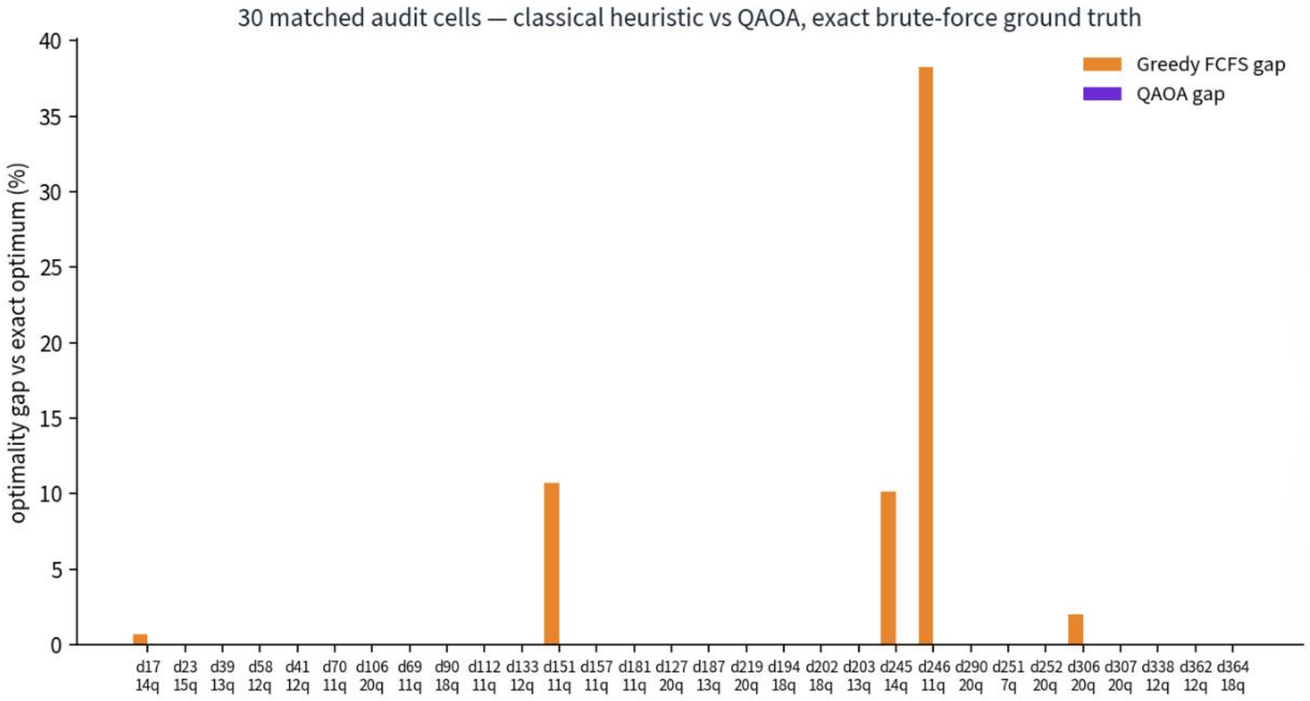


TWO TRACKS, ONE REFEREE

TRACK	DAYS	QAOA
Classical c1–c3	0–182	off
Quantum c4–c6	183–364	**on**

Raw cross-track totals are confounded by fleet growth — so **all headline claims come from 30 matched audit cells**, identical QUBOs, exact ground truth.
COMPARISON.md →

30 matched cells. QAOA: 30/30 exact optimum.



38.23%

worst classical gap above optimum — window 982, day 246, 11 qubits. Recurring per-window money, not a year-total spin (audit covers ~2% of 1,460 windows — **caveat printed with the number**).

METHOD

5 cells per chunk × 6 chunks, stratified. All four solvers attack the **identical QUBO**; brute force (2^{20} = 1,048,576 combos) is the referee. Every row: quantum_audit_all.csv →

The price of a QPU solve is qubits. We cut it 84×.

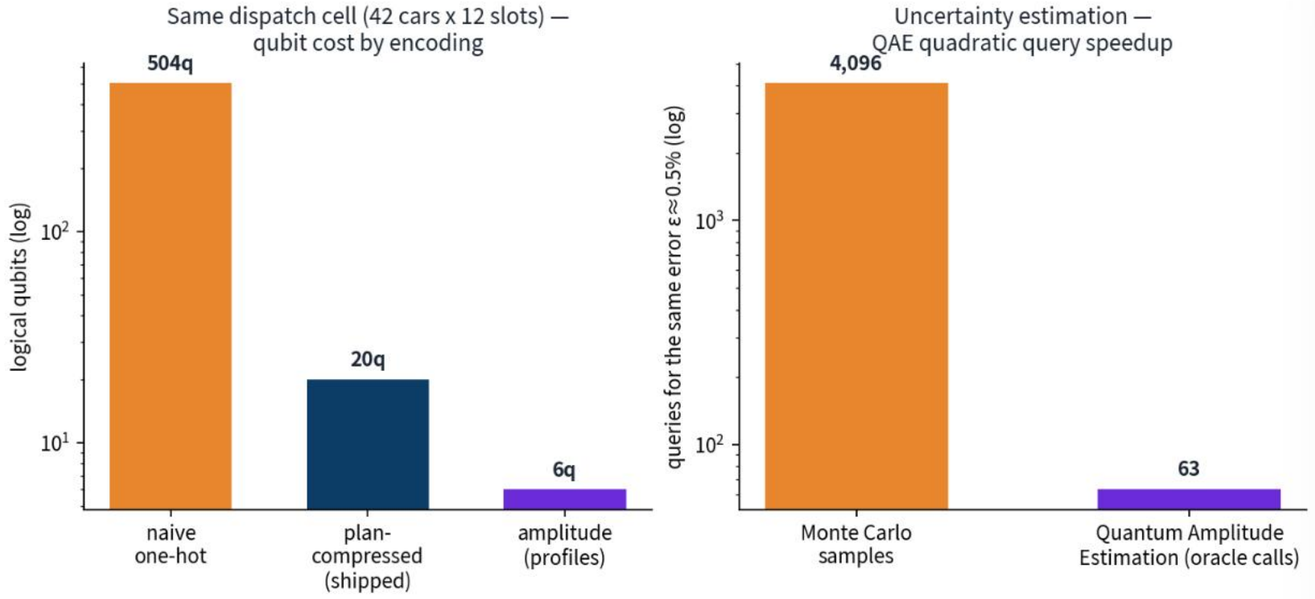
~~504~~
NAIVE ONE-HOT (42 CARS × 12 SLOTS)

→

11–20
PLAN-COMPRESSED QUBO (SHIPPED)

→

6
AMPLITUDE-ENCODED 48-SLOT PROFILE



SIMULATOR TAX — ON THE RECORD

On a laptop statevector sim, contested cells: SA **0.45 s** vs QAOA **96.53 s**. But exact certification of the biggest cell already needs **2^{20}** enumerations and doubles per car — while QAOA scales polynomially. Quality today, scale tomorrow.

Numbers →

63 oracle calls ≈ 4,096 Monte-Carlo samples.

SRC/AMPLITUDE_ENCODING.PY — EXACT STATE PREP

```
# Givens-disentangling prep, Shende–Bullock–Markov bound
rotations.append((j, x >> (j + 1), theta))   # full context
day_profile /= np.linalg.norm(day_profile)      # L2 = a state
amps = np.zeros(64); amps[:48] = profile       # zero-pad

# → 47 Ry + 57 CNOTs, fidelity 1.000000
```

QAE VS MC — THREE ANCHORS, 40 TRIALS EACH

QUANTITY	TRUE	QAE RMSE	ORACLE CALLS
year-mean renewable	9.2%	0.0077	**63**
daily-max renewable	21.2%	0.0102	**63**
midday solar-glut	74.4%	0.0083	**63**

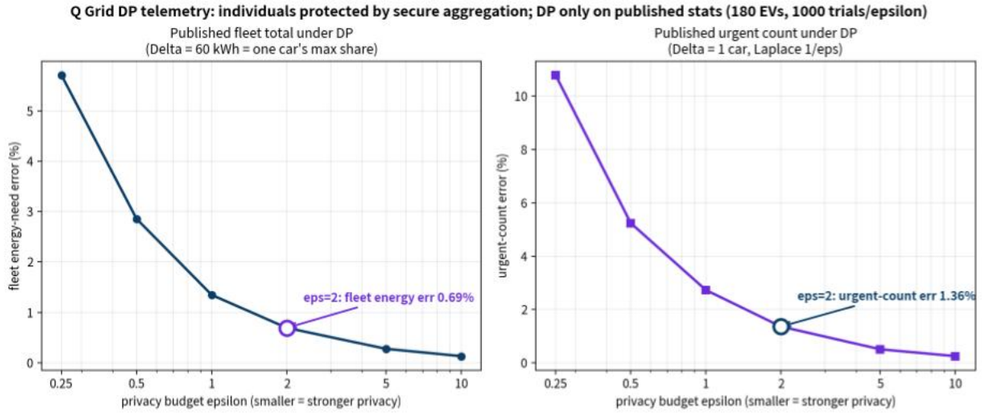
MC needs 4,096 samples to match → **65× query reduction**. JSON: amplitude_encoding.json →

🔬 **Bug-driven confidence:** three quantum bugs (bitmask leak, S_0-basis, L1/L2) each produced a *precisely wrong* signature — we hand-traced n=2 to catch gate leakage and read fidelity 0.045 = $\|p\|^2$ as the normalization clue. The fixed module now agrees with native Qiskit (fidelity 1.0).

Five layers, in the order a driver experiences them.

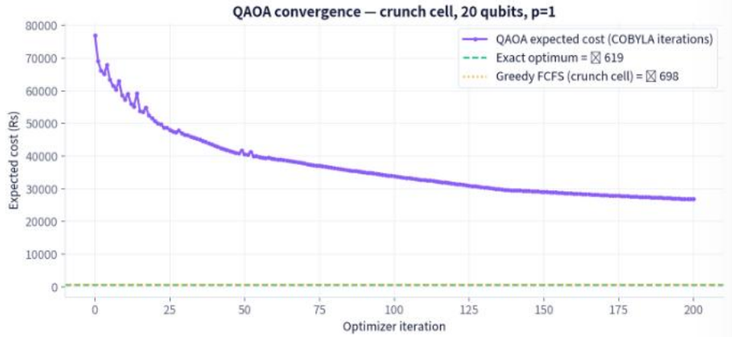
1. **Consent ledger** — nothing flows without explicit opt-in at registration
2. **Ward coarsening** — location coarsened to ward level before leaving the car
3. **Rotating pseudonyms** — trajectories cannot be stitched back together
4. **Differential privacy** — Laplace noise, ε=2 measured: **0.69% fleet error**
5. **Secure aggregation** — the Center sees fleet totals only; exact-sum r = −0.04

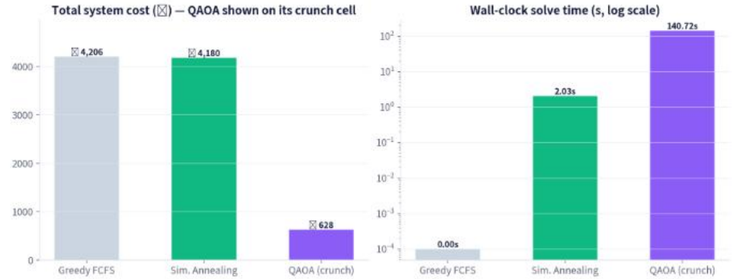
Live benchmark: privacy_audit.json → · code: src/privacy_layer.py →



Proof, not promises.







Gantt — one session per car · **QAOA convergence — p=1, COBYLA** · **4-solver comparison vs exact** · **demo: wait −21.3% vs FCFS · midday renewable 74.4%**

14 · TWENTY LIMITATIONS — EACH WITH A FIX AND A PROOF

We publish our gaps on purpose.

1 · QPU = simulator (no NISQ noise)	2 · laptop QAOA slower than SA	3 · 20-qubit exact ceiling	4 · synthetic demand
5 · telemetry privacy → 5-layer fix	6 · supply uncertainty simplified	7 · no V2G yet (one QUBO term away)	8 · Tokyo-only calibration
9 · no transformer load-flow	10 · human no-shows → deferred queue	11 · amp-enc = data, not variables	12 · QPE depth → AQAE on roadmap
13 · 48→64 zero-padding quantified	14 · single seed (42, pinned)	15 · static ToU → tariff = data input	16 · audit = 2% of windows
17 · QAOA p=1 (already 30/30)	18 · fleet-growth confound → audit	19 · no physical QPU run	20 · residual wait 0.306 slots

Every one has **The limit** / **Why it matters** / **The fix** / **PROOF** pointing at committed code or data — the page cannot drift from reality because reality is committed next to it.
👉 docs/LIMITATIONS.md — and 8 judge counter-answers: docs/COUNTER_ANSWERS.md

15 · THE EIGHT QUESTIONS JUDGES PUSH HARDEST

Counter-answers, with numbers.

"QAOA is slower than SA — why quantum?"

Quality, not time: 30/30 exact optima vs classical gaps to 38.23%. The 96 s is simulator tax; exact certification doubles per car, QAOA doesn't.

"Why not just run the exact solver?"

We do — as referee, not engine. 2^{20} per crunch cell; city-wide (180 EVs) is infeasible. Layered pipeline: FCFS+SA carry load, QAOA hits contested cells.

"Isn't the data synthetic?"

Yes — labeled everywhere. Calibrated: Japanese EV params, Tokyo wards, ToU + tsuyu, 48→180 EVs. Synthesis = full year reproducible in 991 s by any judge.

"How do you avoid stealing driver data?"

Consent ledger → ward coarsening → rotating pseudonyms → DP (ε=2, 0.69% fleet err) → secure aggregation. Audited live in privacy_audit.json.

"What does amplitude encoding buy?"

48-slot profile: 48 wires → 6 qubits, fidelity 1.0. Stacked: 504 → 20 → 6. QAE: 65× fewer queries. Caveat stated: $O(2^n)$ prep gates.

"How do I know the year results are real?"

Six chunked runs, one seed, one aggregator that regenerates every KPI/chart from raw chunks. 991 s re-runnable. Commit history shows each chunk landing.

"What if the fleet doubles tomorrow?"

Size tracks contention, not fleet count: cells stay 11–20 qubits, data stays 6. More chunks, not a redesign — proven by 48→180 in-year.

"What is NOT solved?"

NISQ noise, V2G (one QUBO term), transformer physics. All in LIMITATIONS.md. Measured 38.23% gap + public to-do beats a demo that hides both.

16 · REPRODUCE IT YOURSELF — IN 2 MINUTES

Every number re-derivable. Nothing to believe.

```
# clone & run the demo (seed 42, fully deterministic)
git clone https://github.com/Aghosh-mv/Q-Grid.git
pip install -r requirements.txt
python3 src/qgrid_engine.py         # → results/, charts/
python3 src/amplitude_encoding.py   # → 65× QAE proof
python3 src/qiskit_bridge.py        # → Qiskit cross-check
python3 src/privacy_layer.py        # → privacy audit
# year trial: python3 src/qgrid_year.py (chunks c1–c6)
```

EVERYTHING, LINKED

- 📖 README — submission checklist mapping
- 📊 Year dual-track report + audit
- ⚛️ Amplitude encoding & QAE JSON
- 🛡️ Privacy audit (live)
- 🧾 Engineering log — 6 sections
- 🖥️ Live demo dashboard
- 🎞️ 16-slide pitch deck

17 · THANK YOU

Charge at the optimum.

A working, honest, dual-track quantum prototype — verified by an exact referee, compressed to 6 data qubits, privacy-first, and reproducible in 16 minutes on a laptop.

ⓘ github.com/Aghosh-mv/Q-Grid

👥 JustCode × Qub3RAM.team

⚛ Qiskit 2.6

🌱 Apache-2.0 + addendum